

The Prompt Ladder Assignment

Track: General AI Fluency (Backend / AI Automation Engineering)

Phase: Foundations — Week 2

Baseline (Version 0)

Prompt

"Write backend code for a blog app."

Output (Excerpt)

Here is a simple Python script using Flask to create a blog app:

Python

```
from flask import Flask, jsonify, request

app = Flask(__name__)
posts = []

@app.route('/posts', methods=['GET'])
def get_posts():
    return jsonify(posts)

@app.route('/posts', methods=['POST'])
def create_post():
    data = request.get_json()
    posts.append(data)
    return jsonify({"status": "created"}), 201

if __name__ == '__main__':
    app.run(debug=True)
```

Iteration Notes

1. **What changed in prompt:** None (this is the weak baseline prompt).
2. **What improved in output:** Nothing; it generated a bare-minimum working Flask script.
3. **What still failed:** The code defaulted to Flask instead of FastAPI, used volatile in-memory list storage instead of a real database, lacked authentication, schema validation, and error handling, and offered no modular structure.
4. **What to try next:** Add a clear, specific technical goal (Framework + DB stack).

Version 1: Clear Goal

Added Layer

Clearer Goal

Prompt

Write backend code for a blog app using **FastAPI, PostgreSQL, and SQLAlchemy**.

Output (Excerpt)

Python

```
from fastapi import FastAPI, Depends
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

DATABASE_URL = "postgresql://user:password@localhost/blogdb"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

class Post(Base):
    __tablename__ = "posts"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    content = Column(String)

app = FastAPI()

@app.post("/posts")
def create_post(title: str, content: str):
    db = SessionLocal()
    post = Post(title=title, content=content)
    db.add(post)
    db.commit()
    db.refresh(post)
    return post
```

Iteration Notes

1. **What changed in prompt:** Specified the exact tech stack: FastAPI, PostgreSQL, and SQLAlchemy.
2. **What improved in output:** Stopped generating Flask; switched to FastAPI and integrated real relational database connection models.
3. **What still failed:** The entire app is dumped into a single unstructured script without Pydantic schemas, database sessions are not safely managed via dependency injection (`db.close()` is missing), and there are no endpoints for user management or auth.
4. **What to try next:** Add context about project architecture and domain entity scope.

Version 2: Real Context

Added Layer

Real Context

Prompt

Write backend code for a blog app using FastAPI, PostgreSQL, and SQLAlchemy.

Context: This is a production-oriented REST API that requires user registration/login with password hashing, post management (CRUD operations linked to authenticated users), and dependency-injected asynchronous/sync database session handling.

Output (Excerpt)

Python

```
# Cruft added: Includes full JWT token decoding logic, passlib setup,
# user schemas, post schemas, password hashing helper functions,
# and all endpoints combined into one giant 350-line file.

@app.post("/users/", response_model=UserOut)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    hashed_password = pwd_context.hash(user.password)
    db_user = User(email=user.email, hashed_password=hashed_password)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
```

Iteration Notes

1. **What changed in prompt:** Provided domain requirements (JWT auth, User-Post relationships, safe DB session management via dependencies).
2. **What improved in output:** Replaced crude request parameters with proper Pydantic models (UserCreate, UserOut), implemented Depends(get_db) session management, and established foreign key relationships.
3. **What still failed (Honest Failure):** *This made the code readability worse.* By giving the model full domain context without formatting boundaries, it shoved 350+ lines of authentication logic, models, schemas, and routes into a single monolithic code block, rendering it unmaintainable.
4. **What to try next:** Add a strict structural output format specifying modular folder separation.

Version 3: Specified Output Format

Added Layer

Specified Output Format

Prompt

Write backend code for a blog app using FastAPI, PostgreSQL, and SQLAlchemy.

Context: This is a production-oriented REST API that requires user registration/login with password hashing, post management (CRUD operations linked to authenticated users), and dependency-injected database session handling.

Output Format: Provide the codebase formatted into a clean, modular directory structure breakdown with file paths block-by-block (`database.py`, `models.py`, `schemas.py`, `routers/auth.py`, `routers/posts.py`, `main.py`).

Output (Excerpt)

`database.py`

Python

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
...
```

`models.py`

Python

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True)
    hashed_password = Column(String)
    posts = relationship("Post", back_populates="owner")
```

`routers/posts.py`

Python

```
@router.get("/")
def read_posts(skip: int = 0, limit: int = 100, db: Session =
Depends(get_db)):
    return db.query(Post).offset(skip).limit(limit).all()
```

Iteration Notes

1. **What changed in prompt:** Mandated a modular file-and-folder breakdown (`database.py`, `models.py`, `schemas.py`, `routers/`, etc.).
2. **What improved in output:** Eliminated the monolithic single-file clutter; output was segmented into modular, copy-pasteable backend files mirroring standard FastAPI application architecture.
3. **What still failed:** Left production security gaps: hardcoded database URLs, plain secret keys in JWT configurations, missing exception handling for duplicate email registrations, and missing database migrations.
4. **What to try next:** Add explicit constraints regarding security and environment management.

Version 4: Constraints

Added Layer

Constraints

Prompt

Write backend code for a blog app using FastAPI, PostgreSQL, and SQLAlchemy.

Context: This is a production-oriented REST API that requires user registration/login with password hashing, post management (CRUD operations linked to authenticated users), and dependency-injected database session handling.

Output Format: Provide the codebase formatted into a clean, modular directory structure breakdown with file paths block-by-block (`database.py`, `models.py`, `schemas.py`, `routers/auth.py`, `routers/posts.py`, `main.py`).

Constraints:

- Do NOT hardcode credentials or secrets; use `pydantic-settings` to load from a `.env` file.
- Do NOT write inline SQL or bare `try-except` blocks.
- Use FastAPI's `HTTPException` with explicit status codes (`status.HTTP_401_UNAUTHORIZED`, `status.HTTP_404_NOT_FOUND`, `status.HTTP_400_BAD_REQUEST`).

Output (Excerpt)

`config.py`

Python

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    algorithm: str = "HS256"
    access_token_expire_minutes: int = 30

    class Config:
        env_file = ".env"

settings = Settings()
```

`routers/auth.py`

Python

```
if not db_user or not verify_password(form_data.password,
db_user.hashed_password):
```

```
raise HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Incorrect username or password",
    headers={"WWW-Authenticate": "Bearer"},
)
```

Iteration Notes

1. **What changed in prompt:** Imposed security and code quality constraints (`pydantic-settings` environment isolation, explicit HTTP exception handling, no hardcoded values).
2. **What improved in output:** Removed hardcoded connection strings, introduced `pydantic-settings` for `.env` management, and replaced generic HTTP exceptions with standard HTTP status constants and meaningful error messaging.
3. **What still failed:** The generated code lacked database migration setup (Alembic configuration) and Container deployment files (`Dockerfile` / `docker-compose.yml`) necessary for local developer onboarding and deployment.
4. **What to try next:** Add quality criteria and verification instructions for local execution and database migrations.

Version 5: Quality Criteria & Verification Requirements

Added Layer

Quality Criteria & Verification Requirements

Prompt

Write backend code for a blog app using FastAPI, PostgreSQL, and SQLAlchemy.

Context: This is a production-oriented REST API that requires user registration/login with password hashing, post management (CRUD operations linked to authenticated users), and dependency-injected database session handling.

Output Format: Provide the codebase formatted into a clean, modular directory structure breakdown with file paths block-by-block (`database.py`, `models.py`, `schemas.py`, `routers/auth.py`, `routers/posts.py`, `main.py`).

Constraints:

- Do NOT hardcode credentials or secrets; use `pydantic-settings` to load from a `.env` file.
- Do NOT write inline SQL or bare `try-except` blocks.
- Use FastAPI's `HTTPException` with explicit status codes.

Quality Criteria & Verification: Include an Alembic setup guide for database migrations, a production-ready `Dockerfile`, and step-by-step verification

commands (pytest structure or curl test sequence) to validate the authentication and CRUD endpoints locally.

Output (Excerpt)

Includes all modular backend code modules, plus:

Dockerfile

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Verification Sequence

Bash

```
# 1. Run Migrations
alembic upgrade head

# 2. Register User
curl -X POST "http://localhost:8000/users/" -H "Content-Type: application/json" -d '{"email":"test@example.com", "password":"secretpassword"}'

# 3. Authenticate & Obtain Token
curl -X POST "http://localhost:8000/token" -data "username=test@example.com&password=secretpassword"
```

Iteration Notes

1. **What changed in prompt:** Added quality criteria demanding containerization support, database migration handling, and verification testing workflows.
2. **What improved in output:** Produced a self-contained, end-to-end repository response that includes production deployment artifacts (Dockerfile) and actionable migration/testing steps.
3. **What still failed:** None for the immediate scope; output reached full production starter readiness.
4. **What to try next:** Final cleanup into a modular, reusable prompt template for developers on this track.

Final Reusable Prompt

Markdown

Role: Senior Python Backend Developer & AI Systems Architect

Task:

Generate a production-ready RESTful API backend using FastAPI, PostgreSQL, and SQLAlchemy ORM.

Context:

The application requires user authentication (JWT tokens with password hashing) and management for a blog domain model (Users -> Posts). Database access must leverage SQLAlchemy ORM models with safe dependency-injected session management (`Depends(get_db)`).

Output Format:

Output the complete solution using a modular directory structure with dedicated code blocks labeled by file paths:

```
├── config.py
├── database.py
├── models.py
├── schemas.py
├── utils.py
├── routers/
│   ├── auth.py
│   └── posts.py
├── main.py
├── Dockerfile
└── README.md (Setup & Verification)
```

Constraints:

1. Environment Isolation: Use `pydantic-settings` to parse configuration variables from `.env`. Never hardcode secrets or connection strings.
2. Data Validation: Use Pydantic v2 schemas for all request inputs and response models.
3. Error Handling: Use explicit `fastapi.HTTPException` with `status` code constants (e.g., `status.HTTP_401_UNAUTHORIZED`).
4. Code Cleanliness: Adhere to PEP 8, typing annotations, and async/sync route standards where appropriate.

Verification & Delivery Requirements:

- Include Alembic initialization and migration commands in `README.md`.
- Provide a standard `Dockerfile` targeting `python:3.11-slim`.
- Supply a sequence of `curl` commands or HTTP test scenarios demonstrating complete authentication and CRUD workflows.